

Tinh luyện và nén cấu trúc dữ liệu

Cao Qinxiang

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Tinh luyện và nén cấu trúc dữ liệu

IOI China candidate team papers / 2008 / Day 1 / Cao Qinxiang / paper.doc

Tóm tắt

Độ phức tạp thời gian và không gian là hai thước đo quan trọng của một cấu trúc dữ liệu. Nhiều khi, cách lưu trữ đơn giản hơn hoặc quy mô lưu trữ nhỏ hơn sẽ kéo theo thuật toán nhanh hơn và ít tốn bộ nhớ hơn. Bài viết trình bày ba thủ pháp thường gặp để biến cấu trúc phức tạp thành cấu trúc gọn hơn: tinh luyện thông tin vô dụng, nén cấu trúc lưu trữ, và gộp thông tin lặp lại.

Từ khóa: tinh luyện cấu trúc; giảm quy mô lưu trữ; nén; DFS order; BFS order; cấu trúc dữ liệu.

1 Dẫn nhập

Cấu trúc dữ liệu giữ vai trò ngày càng lớn trong thi tin học. Một mặt, cấu trúc dữ liệu tốt là nền tảng để hiện thực thuật toán hiệu quả, chẳng hạn dùng cây tìm kiếm cân bằng để duy trì quan hệ thứ tự động. Mặt khác, nhiều bài thi hiện nay gần như trực tiếp yêu cầu thiết kế một cấu trúc dữ liệu hỗ trợ một tập thao tác nhất định, ví dụ các bài *Necklace* của NOI/CEOI.

Khi đánh giá một chương trình, ta thường xét thời gian, bộ nhớ, độ chính xác và độ khó cài đặt. Với cấu trúc dữ liệu, nếu độ khó cài đặt còn kiểm soát được, thì thời gian và bộ nhớ là hai tiêu chí quan trọng nhất.

Ý tưởng chung của bài viết là **hóa phức thành giản**. Một cấu trúc nhỏ và đơn giản thường cho phép thao tác nhanh hơn. Quan trọng hơn, sau khi đổi cách nhìn, ta có thể dùng những công cụ quen thuộc hơn. Chẳng hạn, bài toán trên cây đôi khi được chuyển thành bài toán trên dãy để dùng cây đoạn hoặc Fenwick; bài toán trên đồ thị đôi khi được ép về cây DFS hoặc cây BFS để có thêm tính chất cấu trúc.

Bài viết dùng ba chữ để mô tả các thủ pháp:

- **tinh luyện:** bỏ thông tin không ảnh hưởng tới đáp án;
- **nén:** đổi cách lưu để cấu trúc phức tạp thành cấu trúc đơn giản;
- **thu gọn:** gộp các phần thông tin lặp lại.

2 Giản hóa cấu trúc hai chiều

Cấu trúc hai chiều có hai dạng thường gặp. Dạng thứ nhất là hai chiều đối xứng như ma trận. Dạng thứ hai là một bảng tuyến tính mà mỗi phần tử lại là một cấu trúc một chiều, ví dụ mảng các chuỗi. Hai ví dụ sau minh họa hai hướng xử lý.

2.1 Train Car Sorting

Bài *Ural 1568* cho một hoán vị và một thao tác biến đổi tương đối khó nhìn trực tiếp. Điểm mấu chốt trong lời giải gốc là tìm ra một bất biến và mô tả hoán vị bằng một **ma trận mẹ**. Trong ma trận ấy, các phần tử khác không đọc theo một chiều cho ra hoán vị hiện tại, còn đọc theo chiều khác cho ra dãy tăng.

Trong các ma trận mẹ của một hoán vị, ta xét ma trận tối giản, tức số hàng và số cột không thể giảm thêm. Thuật toán lặp lại:

1. nếu hoán vị đã tăng thì dừng;
2. dựng ma trận mẹ tối giản;
3. đọc lại các hàng theo quy tắc chẵn-lẻ để tạo hoán vị kế tiếp.

Phần chứng minh trong bản gốc cho thấy số bước của thuật toán là tối ưu. Nếu ma trận tối giản có p hàng, một thao tác không thể làm số hàng của ma trận tối giản giảm xuống quá nhanh; chứng minh dùng nguyên lý Dirichlet và tính tối giản của ma trận để dẫn tới mâu thuẫn khi giả sử có lời giải tốt hơn.

Điểm liên quan tới cấu trúc dữ liệu nằm ở cách cài đặt. Nếu lưu cả ma trận, mỗi thao tác có thể tốn $\mathcal{O}(n^2)$ thời gian và bộ nhớ. Nhưng ma trận chỉ có n phần tử khác không; các ô bằng không hoàn toàn là thông tin vô dụng. Vì vậy chỉ cần lưu vị trí của các phần tử khác không. Quy mô lưu trữ giảm xuống $\mathcal{O}(n)$, tổng thời gian trở thành $\mathcal{O}(n \cdot \text{ans})$, đồng thời cũng là chi phí xuất lời giải.

Đây là ví dụ điển hình của **tinh luyện**: bỏ phần tử rỗng của ma trận, giữ lại đúng thông tin có tác dụng với thuật toán.

2.2 CEOI 2007 Necklaces

Bài này yêu cầu xây một thư viện duy trì nhiều chuỗi số nguyên đã biết. Ban đầu chỉ có chuỗi rỗng. Mỗi thao tác tạo chuỗi mới bằng cách thêm hoặc xóa một phần tử ở đầu trái hoặc đầu phải của một chuỗi cũ, hoặc hỏi phần tử đầu trái hay đầu phải.

Nếu lưu từng chuỗi riêng, trong trường hợp xấu có $\mathcal{O}(n)$ chuỗi dài $\mathcal{O}(n)$, dẫn tới bộ nhớ bậc hai. Nhưng mỗi chuỗi mới chỉ khác chuỗi cũ ở một thao tác nhỏ, nên giữa các chuỗi có rất nhiều đoạn chung.

Bản gốc đề xuất cấu trúc **left-right tree**. Ta chia mỗi chuỗi thành hai phần: phần trái lưu trên một trie trái, phần phải lưu trên một trie phải. Một chuỗi s được mô tả bởi bốn con trỏ:

$$\text{left_root}, \quad \text{left_leaf}, \quad \text{right_root}, \quad \text{right_leaf}.$$

Chuỗi thực tế là đường từ left_leaf lên left_root trong cây trái, ghép với đường từ right_root xuống right_leaf trong cây phải. Chuỗi rỗng được biểu diễn bằng các con trỏ gốc đặc biệt.

Thêm phần tử ở đầu trái tương ứng với thêm một nút vào cây trái. Xóa đầu trái tương ứng với dịch một đầu mút trên cây trái; nếu phần trái rỗng thì thao tác chuyển sang cây phải. Các thao tác bên phải đối xứng.

Nhờ gộp các đoạn chung bằng trie, bộ nhớ giảm từ $\mathcal{O}(n^2)$ xuống $\mathcal{O}(n)$. Vấn đề còn lại là tìm nhanh “con kế tiếp trên một đường” trong trie. Phần sau của bài quay lại điểm này bằng cấu trúc cha nhảy.

Ví dụ này là dạng **thu gọn**: thông tin lặp lại giữa các chuỗi được gom vào một cây dùng chung.

3 Giải hóa cấu trúc cây

Cây là cấu trúc quen thuộc và là nền của nhiều cấu trúc dữ liệu khác. Tuy vậy, nhiều bài trên cây khó xử lý trực tiếp. Một cách hữu ích là đổi cây thành dãy, thường qua thứ tự DFS hoặc mã ngược.

3.1 Hide and Seek

Bài chọn lọc tỉnh Chiết Giang 2007 cho một cây, mỗi đỉnh có hai màu. Hai thao tác là đảo màu một đỉnh và hỏi khoảng cách lớn nhất giữa hai đỉnh đen. Đây là dạng bài “cập nhật cục bộ, hỏi thuộc tính toàn cục”; trên đây có thể dùng cây đoạn, nhưng trên cây thì chưa trực tiếp dùng được.

Bản gốc mã hóa cây bằng chuỗi ngoặc thu được từ DFS. Ví dụ một cây có thể được viết thành

$$[A[B[E][F[H][I]]][C][D[G]]],$$

trừ bỏ nhãn đỉnh để còn chuỗi ngoặc. Với hai đỉnh P, Q , lấy đoạn mã ngoặc nằm giữa chúng. Sau khi xóa các cặp ngoặc khớp, nếu còn a dấu đóng và b dấu mở, thì khoảng cách giữa P và Q là $a + b$. Trực giác là: a lần đi lên, rồi b lần đi xuống.

Vì vậy mỗi đoạn ngoặc có thể được mô tả bằng cặp (a, b) , trong đó các cặp ngoặc khớp đã bị triệt tiêu. Khi ghép hai đoạn $S_1(a_1, b_1)$ và $S_2(a_2, b_2)$, có thêm $\min(b_1, a_2)$ cặp bị triệt tiêu:

$$(a, b) = \begin{cases} (a_1 - b_1 + a_2, b_2), & a_2 < b_1, \\ (a_1, b_1 - a_2 + b_2), & a_2 \geq b_1. \end{cases}$$

Để cây đoạn trả lời được đường kính của các đỉnh đen, mỗi nút cây đoạn lưu thêm các đại lượng tốt nhất ở tiền tố/hậu tố:

$$\begin{aligned} &right_plus, \quad right_minus, \\ &left_plus, \quad left_minus, \end{aligned}$$

cùng với dis , là khoảng cách lớn nhất giữa hai đỉnh đen nằm trong đoạn. Khi ghép hai con, dis của đoạn cha là cực đại của dis hai con và hai kiểu ghép qua biên. Các công thức còn lại cũng suy ra từ quy tắc ghép (a, b) .

Sau khi đổi cây thành chuỗi ngoặc, mỗi lần đảo màu một đỉnh chỉ cập nhật vài vị trí đáy của cây đoạn rồi kéo thông tin lên. Truy vấn trả về dis của toàn chuỗi. Ý nghĩa của ví dụ là bước **nén**: cấu trúc cây được ép thành dãy ngoặc, làm cây đoạn trở thành công cụ phù hợp.

3.2 Thống kê trên cây

Bài từ luận văn đội tuyển 2005 của He Lin cho một cây n đỉnh đánh số $1, \dots, n$. Với mỗi đỉnh v , cần tính $t(v)$: số hậu duệ của v có nhãn nhỏ hơn v . Thuật toán thô $\mathcal{O}(n^2)$ quá chậm.

Bản gốc dùng hai dãy:

- dãy DFS thông thường;
- dãy DFS sau khi đảo thứ tự con tại mỗi nút.

Với một đỉnh v , phần sau v trong dãy DFS thứ nhất và phần sau v trong dãy DFS đảo có giao đúng vào vùng hậu duệ của v ; phần còn lại cần hiệu chỉnh bằng các tổ tiên trực tiếp.

Ký hiệu $f(v, S)$ là số đỉnh nhỏ hơn v trong vùng S . Khi đó có công thức:

$$t(v) = f(v, \text{phần sau } v \text{ trong DFS}) + f(v, \text{phần sau } v \text{ trong DFS đảo}) + f(v, P(v)) - v + 1,$$

trong đó $P(v)$ là tập tổ tiên của v .

Các số lượng trong hai dãy có thể tính bằng Fenwick tree giống bài đếm nghịch thế. Số tổ tiên nhỏ hơn v được tính trong một lần DFS bằng cách duy trì Fenwick theo đường từ gốc tới đỉnh hiện tại. Tổng thời gian là $\mathcal{O}(n \log n)$.

Điểm tinh tế là ta không còn lưu trực tiếp quan hệ hậu duệ trong cây. Thay vào đó, thông tin cần thiết được ép vào hai dãy DFS và một truy vấn tổ tiên.

3.3 Bài toán còn lại của Necklaces

Trong cấu trúc left-right tree, cần hỗ trợ hai truy vấn trên một đường trong cây:

- biết hai đầu đường, tìm cha của đầu dưới;
- biết hai đầu đường, tìm con của đầu trên nằm trên đường.

Truy vấn thứ hai không hoàn toàn tầm thường khi số phần tử lớn. Bản gốc dùng cấu trúc “siêu cha”, tức bảng nhảy tổ tiên. Với mỗi nút x , lưu độ sâu $d(x)$ và

$$up(x, k) = \text{tổ tiên thứ } 2^k \text{ của } x.$$

Khi biết đầu trên $root$ và đầu dưới $leaf$, con của $root$ trên đường tới $leaf$ là tổ tiên của $leaf$ ở độ sâu $d(root) + 1$. Truy vấn này làm được trong $\mathcal{O}(\log n)$, và mỗi lần thêm nút cũng cập nhật bảng nhảy trong $\mathcal{O}(\log n)$.

Ở đây ta tiếp tục thấy tinh thần tinh luyện: bỏ qua toàn bộ thông tin hậu duệ không cần thiết, chỉ giữ các mốc tổ tiên theo lũy thừa hai.

4 Giải hóa cấu trúc đồ thị

Đồ thị tổng quát phức tạp hơn cây rất nhiều. Vì vậy một chiến lược thường gặp là dùng DFS hoặc BFS để đặt đồ thị vào một cấu trúc đơn giản hơn.

4.1 Network Attack

Bài *Ural 1557* cho một đồ thị vô hướng liên thông. Cần đếm số cách xóa hai cạnh làm đồ thị mất liên thông. Trước hết tìm tất cả cầu. Nếu có s cầu trong m cạnh, mọi cặp gồm một cầu và một cạnh khác đều thắng; sau đó co các cầu để còn đồ thị không có cầu, rồi đếm các cặp cạnh không cầu mà xóa đi làm đồ thị tách.

Bản gốc dùng cây DFS. Trong đồ thị vô hướng, cây DFS không có cạnh ngang; mọi cạnh ngoài cây đều là cạnh ngược nối một đỉnh với tổ tiên. Với một đỉnh i , gọi $T(i)$ là tập đỉnh trong cây con của i , $R(i)$ là tập tổ tiên của i , $s(i)$ là số cạnh nối $T(i)$ với $R(i)$, và $t(i)$ mô tả tổ tiên cao nhất liên quan tới các cạnh ngược từ $T(i)$.

Sau khi bỏ cầu, một 2-cut có hai dạng:

- nếu phần bị tách là cả cây con $T(i)$, thì cần $s(i) = 2$, và hai cạnh nối $T(i)$ với phần trên chính là cặp cần đếm;
- nếu phần bị tách là hiệu $T(i) \setminus T(j)$, với j nằm trong cây con của i , thì hai cạnh cây nối i, j với cha tương ứng tạo thành cặp, khi các cạnh ngược của hai vùng có cùng tập liên hệ lên tổ tiên.

Các điều kiện cụ thể được kiểm tra trong DFS bằng các giá trị đếm cạnh nối lên tổ tiên và độ sâu cao nhất của cạnh ngược. Điều quan trọng ở đây là đồ thị vô hướng được phân tích thông qua cây DFS; tính “không có cạnh ngang” làm cho mọi phần bị tách có dạng cây con hoặc hiệu của hai cây con.

4.2 Networking the Iset

Bài *Ural 1569* yêu cầu chọn một cây khung của đồ thị sao cho đường kính của cây khung nhỏ nhất. Đây là một bài đồ thị có vẻ khó nếu tìm trực tiếp trên mọi cây khung.

Ta xét tâm của một cây. Nếu đường kính chẵn, tâm là một đỉnh duy nhất; nếu đường kính lẻ, tâm là hai đỉnh kề nhau. Với một gốc v , mọi cây BFS gốc v có cùng độ sâu lớn nhất. Nếu cây tối ưu có đường kính chẵn và tâm là p , thì bất kỳ cây BFS gốc p nào cũng có đường kính không lớn hơn cây tối ưu. Nếu đường kính lẻ và hai tâm là p, q , cần một cây BFS gốc p sao cho mọi đỉnh ở tầng sâu nhất đều nằm dưới cùng một con q ; khi đó đường kính là $2d - 1$, thay vì $2d$.

Vì vậy ta thử từng đỉnh v_i làm gốc BFS, lấy độ sâu d_i . Nếu tồn tại một con của gốc chứa toàn bộ các đỉnh ở tầng sâu nhất, đường kính ứng viên là $2d_i - 1$; nếu không, là $2d_i$. Chọn cây BFS có giá trị nhỏ nhất. Với $n \leq 100$, có thể tính khoảng cách mọi cặp bằng Floyd rồi duyệt/kiểm tra các cây BFS trong $\mathcal{O}(n^3)$.

Hai ví dụ đồ thị cho thấy khác biệt giữa DFS và BFS. DFS tạo cây trong đó cạnh ngoài cây chỉ đi lên tổ tiên, nên rất mạnh cho bài liên thông và cắt. BFS tạo cấu trúc theo tầng, nên phù hợp với bài khoảng cách.

5 Kết luận

Các ví dụ trên cho thấy “tinh luyện”, “nén” và “thu gọn” đều xoay quanh hai chữ: giảm và giản. Giảm là giảm lượng thông tin phải lưu; giản là đưa dữ liệu về cấu trúc dễ thao tác hơn.

Khi một thuật toán đang lưu thông tin vô dụng, như ô trống của ma trận, quan hệ hậu duệ không cần thiết của cây, hoặc các phần chuỗi lặp lại, ta nên tìm cách bỏ hoặc gộp nó. Khi dữ liệu quá lộn xộn, ta nên tổ chức lại thành cấu trúc có nhiều tính chất hơn, chẳng hạn cây DFS, cây BFS, dãy DFS hoặc chuỗi ngoặc.

Do đó hóa phức thành giản không chỉ là thủ pháp tối ưu hóa, mà còn là một cách tư duy để tìm đường vào bài toán.

References

- [1] Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi tin học*.
- [2] Mark Allen Weiss, *Data Structures and Algorithm Analysis*.
- [3] Bộ đề và lời giải chọn đội tuyển tỉnh Chiết Giang 2007.
- [4] He Lin, luận văn đội tuyển quốc gia 2005.
- [5] Timus Online Judge.

Ghi chú về nguồn

Bản DOC gốc dài 52 trang và kèm nhiều hình, bảng, đề bài đầy đủ trong phụ lục. Các hình được mô tả bằng lời; các phát biểu đề và phần phụ lục dài được rút gọn để giữ trọng tâm ở kỹ thuật cấu trúc dữ liệu. Công thức bị nhúng dưới dạng đối tượng Word cũ được khôi phục theo ngữ cảnh thuật toán khi cần.